Hướng dẫn pipeline: Agent browser → Google Sheets → Superset

Tổng quan kiến trúc

Pipeline gồm ba khối chính:

1. **Agent + browser automation**: Một agent (LangChain / LlamaIndex / custom Python) dùng trình duyệt tự động (Playwright/Selenium) hoặc HTTP client để crawl/parse dữ liệu từ web.
2. **Google Sheets làm data hub**: Agent ghi dữ liệu đã chuẩn hóa vào một hoặc nhiều Google Sheet thông qua Google Sheets API hoặc thư viện gspread/Google client Python.[1][2]
3. **Apache Superset BI**: Superset kết nối trực tiếp tới Google Sheets (thông qua connector Google Sheets/Shillelagh) và dùng sheet như một nguồn dữ liệu để tạo dataset, chart và dashboard.[3][4]

Mục tiêu tài liệu:

- Thiết kế kiến trúc tổng thể.
- Chi tiết các bước cấu hình Google Sheet, API, agent.
- Hướng dẫn kết nối Google Sheets vào Superset và tạo báo cáo.

Chuẩn bị công nghệ

Thành phần bắt buộc

- **Ngôn ngữ backend**: Python (dễ dùng với Google API + ecosystem agent như LangChain).
- **Thư viện Google**:
  - google-api-python-client, google-auth-httplib2, google-auth-oauthlib hoặc gspread để đọc/ghi Google Sheets.[5][2][1]
- **Agent framework (tuỳ chọn)**:

    - LangChain / LlamaIndex / trực tiếp viết Python class Agent.
- **Browser automation (nếu cần render JS)**:
    - Playwright hoặc Selenium.
- **Apache Superset**: Đã cài (Docker hoặc bare-metal) với phiên bản hỗ trợ Google Sheets connector/shillelagh.[4]

Quyền truy cập Google Cloud

- Một **Google Cloud Project**.
- Đã **enable Google Sheets API** (và nếu dùng file từ Drive private thì enable thêm Google Drive API).[6][1]
- Có **Service Account** hoặc OAuth client phù hợp.

Bước 1: Thiết kế mô hình dữ liệu trong Google Sheets

Trước khi code agent, cần chốt trước schema của sheet để pipeline ổn định.

Nguyên tắc thiết kế sheet

- Mỗi sheet tab tương ứng với **một bảng logic** (ví dụ `orders`, `prices`, `news`…).
- Mỗi dòng là **một bản ghi**.
- Hàng đầu tiên là **header** với tên cột không dấu, snake_case, không trùng, ví dụ:
    - `id`, `timestamp`, `source_url`, `title`, `value`.
- Dùng kiểu dữ liệu thống nhất per cột (date, number, text) để Superset đoán schema tốt hơn.[3]

Ví dụ schema sheet `web_metrics`

- `timestamp` (datetime ISO hoặc `YYYY-MM-DD HH:MM:SS`).
- `source` (domain / hệ thống).
- `metric_name` (ví dụ `visits`, `signups`).
- `metric_value` (number).
- `meta_json` (optional, string JSON dump nếu cần thêm field phụ).

Bước 2: Cấu hình Google Sheets API & Service Account

2.1 Tạo project và enable API

1. Vào Google Cloud Console, tạo project mới hoặc dùng project có sẵn.[2][1]
2. Vào **APIs & Services** → **Library**, bật **Google Sheets API** (và Google Drive API nếu cần).[1][6]

2.2 Tạo Service Account và key

1. Trong **APIs & Services** → **Credentials** → **Create credentials** → **Service account**.
2. Đặt tên, quyền tối thiểu `Editor` cho project (đủ để ghi sheet).[6][1]
3. Vào service account, tab **Keys** → **Add key** → **JSON**, tải file JSON key (ví dụ `credentials.json`).[6]

2.3 Share Google Sheet cho service account

1. Mở Google Sheet sẽ dùng làm data hub.
2. Bấm **Share**, thêm email của service account trong JSON key, cho quyền **Editor**.
3. Ghi lại **spreadsheetId** (phần ID trong URL sheet) để dùng trong code.[5]

Bước 3: Code Python kết nối Google Sheets

3.1 Khởi tạo client Google Sheets

Sử dụng `googleapiclient` với service account:

```
from googleapiclient.discovery import build
from google.oauth2.service_account import Credentials

SCOPES = ["https://www.googleapis.com/auth/spreadsheets"]
SERVICE_ACCOUNT_FILE = "credentials.json"  # đường dẫn đến file JSON

credentials = Credentials.from_service_account_file(
```

```
    SERVICE_ACCOUNT_FILE,
    scopes=SCOPES,
)

service = build("sheets", "v4", credentials=credentials)
SPREADSHEET_ID = "<spreadsheet-id>"  # lấy từ URL
```

Đây là pattern chuẩn trong Quickstart chính thức của Google Sheets API.[2][5]

3.2 Hàm append dòng dữ liệu vào sheet

```
def append_rows(sheet_name: str, rows: list[list]):
    body = {"values": rows}
    service.spreadsheets().values().append(
        spreadsheetId=SPREADSHEET_ID,
        range=f"{sheet_name}!A1",
        valueInputOption="RAW",
        insertDataOption="INSERT_ROWS",
        body=body,
    ).execute()
```

- `rows` là list các bản ghi, mỗi bản ghi là list giá trị theo đúng thứ tự cột.
- `valueInputOption="RAW"` để ghi nguyên value, để format ở Superset.[5]

3.3 Kiểm thử ghi dữ liệu mẫu

```
sample_rows = [
    ["2026-02-27 10:00:00", "example.com", "visits", 123, "{}"],
]
append_rows("web_metrics", sample_rows)
```

Sau khi chạy, mở sheet kiểm tra đã có dòng mới hay chưa.

Bước 4: Thiết kế Agent browser lấy dữ liệu

Có hai hướng chính: dùng **HTTP client** + **parse HTML/JSON** nếu API/public endpoint, hoặc dùng **browser headless** nếu cần render/bypass JS.

4.1 Skeleton Agent đơn giản (Python class)

```python
class WebCrawlAgent:
    def __init__(self, sheets_client):
        self.sheets_client = sheets_client

    def crawl_one_source(self) -> list[list]:
        # TODO: gọi requests / Playwright để lấy dữ liệu,
        # parse và trả list[list] theo schema web_metrics
        raise NotImplementedError

    def run(self):
        rows = self.crawl_one_source()
        if rows:
            append_rows("web_metrics", rows)
```

Trong thực tế có thể wrap logic này vào LangChain **Tool** + **Agent** để điều phối nhiều nguồn và thêm logic quyết định.

4.2 Dùng LangChain Browser/Requests tool (tùy chọn)

- Tạo tool dùng `requests` hoặc Playwright để fetch HTML/api.
- Tạo tool khác ghi dữ liệu vào Google Sheets bằng hàm `append_rows`.
- Dùng một Agent (ReAct / ToolCalling) để quyết định gọi tool nào, thứ tự, retry…

Ý tưởng là mọi IO (web, sheet) đều được đóng gói thành **tools** mà agent có thể gọi.

Bước 5: Lên lịch chạy agent định kỳ

Để dữ liệu trong sheet luôn mới, cần cơ chế schedule:

- Cron job (Linux) chạy script Python mỗi X phút/giờ.
- Hoặc dùng n8n / Airflow / GitHub Actions để gọi script định kỳ.
- Đảm bảo idempotent hoặc có cột `timestamp` để phân biệt lần crawl.

Bước 6: Kết nối Google Sheets vào Superset

Apache Superset hỗ trợ Google Sheets thông qua driver **Shillelagh** / connector Google Sheets.[7][4]

6.1 Cài đặt dependency Google Sheets cho Superset

Nếu tự host Superset (Docker), cần thêm extra:[4]

```
pip install "apache-superset[gsheets]"
```

Hoặc theo hướng dẫn của Superset: thêm `shillelagh[gsheetsapi]` vào requirements và rebuild container.[8][4]

6.2 Tạo kết nối "Google Sheets" trong Superset

Tùy phiên bản, có 2 cách phổ biến:

1. **Dùng connector Google Sheets sẵn trong UI** (Superset mới hoặc Preset Cloud):
   - Vào `Data` → `Connect Google Sheet`.
   - Dán URL share của Google Sheet (thường ở chế độ `Anyone with link can view` nếu dùng public connector).[9][3]
   - Đặt tên database (ví dụ `web_metrics_gsheet`) và lưu.
2. **Dùng connection string gsheets://**:
   - Vào `Settings` → `Database Connections` → `+ Database`.
   - Connection string: `gsheets://`.
   - Phần cấu hình chi tiết tham chiếu docs Google Sheets của Superset.[8][4]

Việc dùng Shillelagh cho phép query Google Sheets như bảng SQL trong Superset.[7][4]

Bước 7: Tạo Dataset từ Google Sheets

Sau khi kết nối database Google Sheets:

1. Vào `Data` → `Datasets` → `+ Dataset`.[10][3]
2. Chọn **Database** là kết nối Google Sheets vừa tạo (ví dụ `web_metrics_gsheet`).[3]
3. Chọn **Schema** / **Table** tương ứng với tab `web_metrics` trong Sheet.[3]
4. Lưu lại dataset.

Bây giờ, tab `web_metrics` của Google Sheets đã xuất hiện như một bảng dữ liệu trong Superset và có thể dùng để tạo chart.[3]

Bước 8: Tạo chart và dashboard trong Superset

8.1 Tạo chart từ dataset Google Sheets

1. Vào `Charts` → `+ Chart`.[11][3]
2. Chọn dataset `web_metrics` (Google Sheets).[3]
3. Chọn loại chart, ví dụ **Line chart** cho metric theo thời gian.
4. Cấu hình:
    - **Time column**: `timestamp`.
    - **Metric**: `SUM(metric_value)`.
    - **Group by**: `metric_name` hoặc `source`.
5. Bấm **Run** để xem preview, sau đó **Save** chart.[11][10]

8.2 Tạo dashboard

1. Vào `Dashboards` → `+ Dashboard`.
2. Đặt tên dashboard (ví dụ Web Metrics Dashboard).[10]
3. Mở dashboard, bấm **Edit**, thêm các chart vừa tạo.[11][3]
4. Sắp xếp layout, cấu hình filter (ví dụ filter theo `source`, `metric_name`).

Trong các tutorial Superset, quy trình này là pattern chuẩn: connect DB → tạo dataset → tạo chart → gộp vào dashboard.[10][11][3]

Bước 9: Đảm bảo dữ liệu cập nhật realtime/gần realtime

Để dashboard luôn có dữ liệu mới khi agent ghi vào sheet:

- Khi Superset query Google Sheets, nó sẽ đọc dữ liệu mới nhất từ sheet thông qua connector Shillelagh.[7][4]
- Có thể cấu hình **cache TTL** nhỏ trong Superset nếu cần gần realtime.
- Lịch chạy agent cần tương thích với expectation (5 phút, 15 phút, 1 giờ…).

Bước 10: Mở rộng & hardening

Một số hướng mở rộng cho production:

- **Validation trước khi ghi**: agent validate schema, type, range value trước khi append để tránh bẩn dữ liệu.
- **Logging & monitoring**: log số bản ghi mỗi lần crawl, error rate, lưu vào log riêng.
- **Tách staging & prod sheet**: agent ghi vào sheet staging, có bước kiểm/clean rồi copy sang sheet prod dùng cho Superset.
- **Chuyển dần sang warehouse**: khi volume lớn, có thể chuyển backend thành Postgres/BigQuery, nhưng vẫn giữ cùng kiến trúc agent → storage → Superset.

Checklist triển khai nhanh

- [ ] Tạo Google Cloud project, enable Sheets API, tạo service account, tải credentials JSON.[1][2][5]
- [ ] Tạo Google Sheet, thiết kế schema tab `web_metrics`, share cho service account.[5][3]
- [ ] Viết script Python kết nối Google Sheets và test ghi dữ liệu mẫu.[2][5]
- [ ] Bọc logic crawl web vào agent, push data vào sheet theo schema cố định.
- [ ] Thiết lập cron/n8n/Airflow để chạy agent định kỳ.
- [ ] Cài `apache-superset[gsheets]` và cấu hình database Google Sheets trong Superset.[8][4][7]

- [ ] Tạo dataset từ tab `web_metrics` và build chart, dashboard trong Superset.[11][10][3]

Với kiến trúc này, có thể nhanh chóng dựng một hệ thống auto crawl → Google Sheets → Superset phục vụ báo cáo mà không cần dựng data warehouse phức tạp ngay từ đầu.[4][7][3]

References

1. How to Automate Google Sheets with Python? - GeeksforGeeks - Your All-in-One Learning Portal: GeeksforGeeks is a comprehensive educational platform that empowers...
2. Python quickstart | Google Sheets
3. How to Create Charts from Google Sheets in Superset| Sabbirz ... - Learn how to connect a Google Sheet as a data source in Apache Superset. This step-by-step guide sho...
4. Google Sheets - Superset - Apache Software Foundation - Common error messages you may encounter when connecting to or querying Google Sheets, along with the...
5. How to Use Google Sheets API with Python: Read, Write, Update, Delete - In this detailed tutorial, learn how to use the Google Sheets API through Google Cloud Platform (GCP...
6. Automate Google Sheets With Python - YouTube - Secure your business effortlessly with a 3-month NordPass trial! Use ”'techwithtim”' activation code...
7. Accessing APIs with Superset - Preset - Connect Superset to REST APIs and non-traditional data sources like Google Sheets using Shillelagh, ...
8. Getting Started - Connect Superset To Google Sheets - Preset - This tutorial shows you how to connect your local deployment of Apache Superset with Google Sheets, ...
9. Apache Superset Google Sheets: Build Charts Without Code [Part 1] - *Resources & Links*
   *For the detailed written steps, check out the associated blog post on sabbirz...
10. Creating Your First Dashboard - Apache Superset - This tutorial targets someone who wants to create charts and dashboards in Superset. We'll show you ...

11. Apache Superset: Create Dataset & Table Chart [Part 2] -
    Resources & Links
    *For the detailed written steps, check out the associated blog post on sabbirz...